Practical Experiment Worksheet–7

Design and Performance Analysis of Materialized Views

---

1. Aim of the Experiment

To design and implement **simple views, complex views, and materialized views**, and to compare their execution time and performance in order to understand their role in query optimization and system efficiency.

---

2. Objectives of the Experiment

The objectives of this experiment are:

1. To understand the concept of views and materialized views.
2. To create simple and complex views using SQL.
3. To implement materialized views for storing precomputed data.
4. To analyze and compare execution time of different views.
5. To evaluate performance improvements using materialized views.
6. To understand real-world applications in enterprise systems.

---

3. Software / Tools Required

- Oracle Database Express Edition (Oracle XE)
- PostgreSQL Database
- Oracle SQL Developer / SQL*Plus
- pgAdmin (for PostgreSQL)
- Computer system with database access

---

4. Theory

Structured Query Language (SQL) is used to manage and manipulate relational databases. In large-scale systems, executing complex queries repeatedly can degrade performance.

🔹 Views

A view is a **virtual table** based on a SQL query. It does not store data physically.

Types of Views:

1. Simple View

- Based on a single table
- No aggregation or joins
- Faster and easy to use

---

2. Complex View

- Uses joins, GROUP BY, aggregate functions
- Computed at runtime
- Slower compared to simple views

🔹 **Materialized View**

A materialized view stores the result of a query **physically in the database**.

Features:

- Faster query execution
- Reduces computation time
- Requires periodic refresh

🔹 **Performance Concept**

- Simple View → Direct access
- Complex View → Recalculated each time
- Materialized View → Precomputed results

---

5. Procedure

Step 1
Start the system and open Oracle SQL Developer.

Step 2
Open a new SQL Worksheet.

Step 3
Drop existing table and views (if any).

Step 4
Create Employee table.

Step 5
Insert sample data into the table.

Step 6
Create a simple view.

Step 7
Create a complex view using aggregation.

Step 8
Create a materialized view.

Step 9
Execute queries on all views.

Step 10
Measure and compare execution time.

---

6. SQL Commands Used in the Experiment

Step 1: Drop Existing Objects

```
BEGIN
   EXECUTE IMMEDIATE 'DROP MATERIALIZED VIEW mv_employee';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
   EXECUTE IMMEDIATE 'DROP VIEW complex_view';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
   EXECUTE IMMEDIATE 'DROP VIEW simple_view';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
   EXECUTE IMMEDIATE 'DROP TABLE Employee';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/
```

Step 2: Create Employee Table

```
CREATE TABLE Employee (
   EmpID NUMBER PRIMARY KEY,
   EmpName VARCHAR2(50),
   Salary NUMBER,
   DeptID NUMBER
);
```

Step 3: Insert Data into Table

```
INSERT INTO Employee VALUES (1, 'Aman', 30000, 10);
INSERT INTO Employee VALUES (2, 'Riya', 40000, 20);
INSERT INTO Employee VALUES (3, 'Karan', 50000, 10);
INSERT INTO Employee VALUES (4, 'Neha', 35000, 30);
INSERT INTO Employee VALUES (5, 'Rahul', 60000, 20);
INSERT INTO Employee VALUES (6, 'Priya', 45000, 10);

COMMIT;
```

Step 4: Create Simple View

```
CREATE VIEW simple_view AS
SELECT EmpID, EmpName, Salary
FROM Employee;
```

Step 5: Create Complex View

```
CREATE VIEW complex_view AS
SELECT DeptID,
     COUNT(*) AS total_emp,
     AVG(Salary) AS avg_salary
FROM Employee
GROUP BY DeptID;
```

Step 6: Create Materialized View

```
CREATE MATERIALIZED VIEW mv_employee
BUILD IMMEDIATE
REFRESH COMPLETE
AS
SELECT DeptID,
     COUNT(*) AS total_emp,
     AVG(Salary) AS avg_salary
FROM Employee
GROUP BY DeptID;
```

Step 7: Execute Queries

```
SELECT * FROM simple_view;
SELECT * FROM complex_view;
SELECT * FROM mv_employee;
```

Step 8: Performance Measurement

```
SET TIMING ON;

SELECT * FROM simple_view;
SELECT * FROM complex_view;
SELECT * FROM mv_employee;

SET TIMING OFF;
```

Step 9: Refresh Materialized View

```
BEGIN
   DBMS_MVIEW.REFRESH('MV_EMPLOYEE');
END;
/
```

---

7. Output

The output includes:

- Employee data from simple view
- Aggregated department data from complex view
- Precomputed results from materialized view

👉 Execution time comparison shows:

- Simple view → fast
- Complex view → slower
- Materialized view → fastest

(Similar to output representation shown in Experiment-5 screenshot format)

	DEPTID	TOTAL_EMP	AVG_SALARY
1	10	3	1666.666666666664
2	20	2	50000
3	30	1	35000

Download Execution time: 0.001 seconds

	EMPID	EMPNAME	SALARY
1	1	Aman	30000
2	2	Riya	40000
3	3	Karan	50000
4	4	Neha	35000
5	5	Rahul	60000
6	6	Priya	45000

---

8. Observations

1. The Employee table was created successfully.
2. Data was inserted correctly into the table.
3. Simple view retrieved data quickly.
4. Complex view required additional computation time.
5. Materialized view provided faster query execution.
6. Performance improved significantly for repeated queries.
7. Materialized view required refresh for updated data.

---

9. Result

The experiment was successfully performed.
Simple, complex, and materialized views were created and analyzed. It was observed that **materialized views significantly improve performance** by storing precomputed results, making them highly useful in large-scale enterprise applications.